Software Requirements Specification (SRS)

Project: Smart Parking Platform Company:

Software Corp.

Version:2.1

Date: 11.05.2026

INTRODUCTION

Purpose

This document defines the system, functional, and non-functional requirements for the Smart Parking Platform developed by Software Corp.

Scope In-Scope:

- Real-time parking availability tracking across connected parking garages.
- Interactive map visualization and integrated navigation assistance.
- Space reservation, digital payment processing, and transaction receipts.
- Garage operator management dashboard with occupancy analytics and pricing controls.

Out-of-Scope:

- On-street municipal metered parking spaces, limited strictly to commercial parking garages.
- Direct vehicle repair, towing services, or valet dispatching.

Vision and Company Background

Company Introduction: Software Corp. is a premier software engineering studio with development capabilities, tasked with designing and deploying a scalable urban infrastructure system.

How Acquired: Commissioned as a digital transformation initiative to modernize private garage utilization, mitigate downtown traffic bottlenecks, and automate parking operator revenue streams.

Project Overview: A unified mobile and web platform that connects drivers to live garage spot vacancies, enables frictionless advance reservations and payments, and supplies garage operators with real-time occupancy monitoring and dynamic pricing tools.

Market Research and Landscape Existing Landscape:

- ParkWhiz: Reliable reservation tools but often lack real-time per-stall sensor synchronization and live turn-by-turn guidance to exact available levels or stalls.
- ParkMobile and PayByPhone: Strong transactional and street-meter footprint, but fragmented garage operator dashboards and minimal live occupancy optimization.

Key Differentiators:

- Real-time IoT sensor telemetry showing live stall availability per floor rather than simple aggregate estimates.
- Predictive departure modeling to hold spots right as previous occupants exit.
- Automated turnstile and gate license plate recognition (LPR) validation paired directly with in-app digital passes.

OVERALL DESCRIPTION Users:

- Drivers and Commuters: Locating, reserving, and paying for garage parking.
- Garage Operators: Managing stall inventory, setting dynamic rates, and viewing facility flow.
- City Transportation Officials: Accessing aggregated traffic congestion and occupancy trends.
- System Administrators and Support: Managing platform health, access control, and dispute resolutions.

System Environment:

- Web Application: Responsive single-page web portal for the operator management portal, analytics dashboard, and desktop booking interface.
- Mobile Applications: Cross-platform native mobile apps (iOS and Android) for end-user turn-by-turn navigation, stall reservations, and mobile wallet payments.
- Backend Environment: Cloud-native microservices architecture interfacing with relational databases, external mapping providers, and payment gateways.

Constraints:

- Strict compliance with cardholder payment processing.

• Dependence on third-party mapping APIs and messaging gateways.

• Compliance with local municipal parking ordinances and data privacy standards.

Assumptions:

• Garages possess reliable internet connectivity and entry or exit sensors.

• Users possess GPS-enabled mobile smartphones with active cellular data connections.

FUNCTIONAL REQUIREMENTS

*FR1: The system shall allow drivers to register accounts, authenticate securely, and store payment profiles.

FR2: The system shall render an interactive map displaying parking garages with live open stall counts.

FR3: The system shall allow users to reserve a parking stall for a specified date, arrival window, and duration.

FR4: The system shall securely tokenize payment data and charge users via integrated credit card and digital wallet gateways.

FR5: The system shall generate digital booking passes containing verifiable QR codes and license plate identifiers.

FR6: The system shall provide turn-by-turn route navigation from the user's current GPS location directly to the garage entrance.

FR7: The system shall provide an administrative dashboard allowing operators to configure garage floorplans, total capacities, and hourly rates.

*FR8: The system shall generate real-time visual alerts when garage occupancy exceeds 95% capacity.

FR9: The system shall send push notifications 15 minutes prior to the expiration of an active reservation.

FR10: The system shall allow drivers to extend reservation times dynamically from the mobile app, provided space is available.

NON-FUNCTIONAL REQUIREMENTS

1 (Scalability): The system shall handle at least 1,000 concurrent active users and 100 simultaneous reservations per second during peak hours.

2 (Latency): Interactive map stall availability updates must reflect across client interfaces within 1.0 second of physical sensor triggers.

3 (Availability): The core reservation and payment service shall maintain 99.95% annual uptime.

4 (Security): All API traffic must enforce TLS 1.3 encryption, and sensitive customer data must be encrypted at rest using AES-256.

USE CASES

Use Case 1: Driver Account Registration

Actor: Driver

Precondition: User has installed the mobile app and has an active network connection.

Steps:

1. Driver opens the app and selects Sign Up.

2. Driver enters name, email, phone number, vehicle license plate, and password.

3. System validates formatting and verifies the email address via a one-time code.

4. Driver confirms verification code.

Postcondition: User account is created and stored in the database.

Use Case 2: View Real-Time Garage Availability

Actor: Driver

Precondition: Driver is authenticated and has GPS location permissions enabled.

Steps:

1. Driver opens the interactive map view.

2. System shows garages within a 5-mile radius.

3. System displays pins color-coded by open capacity.

4. Driver taps a garage pin to view rates and remaining spots open.

Postcondition: Garage details and live counts are rendered on screen.

Use Case 3: Reserve a Parking Space

Actor: Driver

Precondition: Driver has selected an eligible garage with available capacity.

Steps:

1. Driver taps Reserve Space.
2. Driver specifies arrival time, expected duration, and selects a vehicle profile.
3. System locks a spot for 5 minutes and calculates the total cost.
4. Driver reviews fee breakdown and taps Confirm and Pay.

Postcondition: Reservation is confirmed and inventory count goes down by one.

Use Case 4: Process Digital Payment

Actor: Driver / Payment Gateway

Precondition: Reservation details are confirmed, user is on checkout screen.

Steps:

1. Driver selects preferred payment method.
2. System gives a charge request to the payment processor.
3. Payment gateway verifies funds and returns an authorization response.
4. System logs the transaction and updates reservation status to Paid.

Postcondition: A digital receipt is generated and emailed to the driver.

Use Case 5: Navigate to Garage Facility

Actor: Driver

Precondition: Driver holds a valid, active reservation.

Steps:

1. Driver views active reservation pass and selects Navigate.
2. System connects to external mapping app.

3. Map app gives the fastest route directly to the designated facility.

Postcondition: Driver receives turn-by-turn navigation to the physical garage entry point.

Use Case 6: Garage Check-In via Digital Pass

Actor: Driver / Garage Scanner System

Precondition: Driver arrives at garage gate during their reservation window.

Steps:

1. Driver scans QR code pass at the gate reader or license plate camera reads vehicle plate.
2. System validates license plate and reservation active timestamp.
3. Entry gate arm raises automatically.
4. System updates reservation state to Checked-In.

Postcondition: Driver successfully enters; garage live occupancy increments.

Use Case 7: Extend Parking Duration

Actor: Driver

Precondition: Driver has an active Checked-In reservation that has not expired.

Steps:

1. Driver receives a 15-minute expiration warning push notification.
2. Driver opens reservation and selects Extend Time by 1 Hour.
3. System verifies garage availability for the extended window.
4. System processes incremental charge and updates scheduled departure time.

Postcondition: Reservation end time extended; gate access updated.

Use Case 8: Garage Exit and Pass Validation

Actor: Driver / Garage Scanner System

Precondition: Vehicle pulls up to exit gate.

Steps:

1. Exit scanner scans QR pass or reads license plate.
2. System verifies no overdue balance exists.
3. Exit gate arm opens.
4. System marks reservation as Completed.

Postcondition: Stall is officially released back to open inventory.

Use Case 9: Cancel Parking Reservation

Actor: Driver

Precondition: Driver holds an upcoming reservation outside the cancellation penalty window.

Steps:

1. Driver selects active reservation under My Bookings.
2. Driver chooses Cancel Reservation.
3. System computes applicable refund policy and releases the held stall.
4. Payment processor issues refund transaction.

Postcondition: Reservation marked Cancelled; inventory count increments.

Use Case 10: Operator Stall Inventory Management

Actor: Parking Facility Operator

Precondition: Operator is logged into the web portal with administrative privileges.

Steps:

1. Operator selects Facility Management, then Floor Plan.
2. Operator marks a specific section as Offline for Maintenance.
3. System saves configuration and updates total rentable stalls across mobile endpoints.

Postcondition: Maintenance stalls are blocked from public search and reservation queries.

Use Case 11: Configure Dynamic Pricing Rules

Actor: Parking Facility Operator

Precondition: Operator is logged in with administrative access.

Steps:

1. Operator navigates to Pricing and Rules Engine.
2. Operator creates a surge rate rule for peak hours or local events.
3. Operator confirms and publishes rule.
4. System updates calculation engine for subsequent quote requests.

Postcondition: New rates apply immediately to future booking requests.

Use Case 12: View Occupancy Analytics and Reports

Actor: Parking Facility Operator

Precondition: Operator dashboard session is active.

Steps:

1. Operator opens Reports and Analytics.
2. Operator selects date range and clicks Generate Report.
3. System queries reservation archives and aggregates peak occupancy, turnover rate, and gross revenue.
4. System renders interactive visual charts and provides a CSV export option.

Postcondition: Operator reviews business utilization metrics.

Use Case 13: System Administrator User Role Assignment

Actor: System Administrator

Precondition: Administrator is logged into internal security administration portal.

Steps:

1. Administrator navigates to User and Organization Management.
2. Administrator searches for a garage facility employee account.
3. Administrator updates assigned role from Viewer to Facility Operator.
4. System updates role-based access control scopes.

Postcondition: Employee gains permissions to adjust facility stall rules.

Use Case 14: Automated Overstay Alert Dispatch

Actor: Automated System Scheduler

Precondition: A vehicle has not exited 30 minutes past the paid reservation window.

Steps:

1. Automated background job checks for open reservations past grace expiration.
2. System flags reservation status as Overstay.
3. System triggers push notification alert to driver prompting payment or immediate exit.
4. System flags vehicle on operator dashboard for enforcement dispatch.

Postcondition: Overstay record created and operator notified.

Use Case 15: Dispute Transaction / Request Refund

Actor: Driver / Support Representative

Precondition: Completed reservation has an associated billing record.

Steps:

1. Driver selects booking history and clicks Report an Issue / Request Refund.
2. Driver inputs reason and submits evidence photo.
3. Support representative reviews ticket in admin console.
4. Support approves dispute; system triggers gateway refund.

Postcondition: Customer receives credit confirmation; ticket logged as resolved.

Executive Summary

The Smart Parking Platform project was set out to design and construct an intelligent, end-to-end parking management and parking spot reservation system. Executed across a 16-week agile lifecycle, the goal that needed to be addressed was campus parking challenges by delivering modules for user authentication, responsive and friendly user interfaces, backend API services, and utilization reporting. The team mapped user's stories, established a comprehensive work breakdown structure (WBS), monitored project risks, and maintained active sprint iterations using Trello and GitHub. The platform reached its baseline minimum viable product objectives within schedule and resource constraints, verifying core functional requirements before official project closure.

Achievements and Shortfalls

Achievements

- Agile Sprint Delivery: Followed and maintained a disciplined 16-week schedule, translating high-level requirements into clear sprint backlogs across the UI, backend, login, and reporting functional tracks.
- Core Functional Baseline: Built and verified core capabilities enabling users to view real-time parking spot availability, register vehicle profiles, execute spot reservations, and access administrative occupancy metrics.
- Version Control Integrity: Followed through with branching and pull request standards in GitHub, ensuring that everything was traceable with commits and versions.

Shortfalls

- IoT Sensor Feeds: Due to hardware acquisition and physical deployment boundaries, space detection relied on synthetic sensor payloads and mock APIs rather than live, campus-wide physical sensors.
- Reporting Feature De-scoping: Advanced predictive occupancy modeling and multi-provider payment gateway integration were deferred to subsequent development phases to preserve core reservation flow stability.
- Mid-Cycle Branch Divergence: Intermittent delays in merging parallel UI and API branches required manual merge conflict resolution during mid-stage sprints.

Lessons Learned

- Deadline schedule: By keeping a strict schedule it taught the team to work better and not be behind on the main goal and finishing the
- API Schema Contracts: Defining rigid API request/response specifications before frontend sprint work begins prevents rework and interface mismatches.

- Continuous Branch Rebasing: Waiting until sprint milestones to integrate feature branches increases merge conflict risk; adopting smaller, continuous branch merges maintains cleaner repository health.
- Story Acceptance Criteria: Clearly defining "Definition of Done" at the individual user story level keeps development tightly focused on core requirements, preventing scope creep.

Recommendations for Future Projects

- Automated CI/CD Integration: Configure automated build and test validation via GitHub Actions from Day 1 to automatically detect syntax errors and regression bugs upon pull request creation.
- Hardware-in-the-Loop Simulation: When physical IoT devices cannot be deployed, build an automated latency and dropout simulator early to stress-test real-time spot updates under poor connectivity.
- Microservice Scalability: For future production scaling, decouple the reporting and reservation backends into containerized services to handle concentrated traffic spikes during peak arrival times.